

Mesh protocol documentation

A technical description of the Mesh blockchain

meshcoin.org

7xy@meshcoin.org

Last updated on 19/9/2026.

1. Introduction

Mesh is a [Proof-of-Work](#) (PoW) blockchain designed with the aim of creating a lightweight, easy to use and accessible cryptocurrency. This document is designed to clearly document how Mesh works, and to provide a clear explanation of the design and design choices used by Mesh. This document does not aim to work as a universal explanation for all cryptocurrencies as the technical details and design choices may differ.

2. Wallets and Balances

In order to allow ownership of a digital currency, an account based system was implemented which consists of a private key, public key and address. The public key and address can be derived from the private key by using [secp256k1](#). However, deriving the private key from the public key is computationally exhaustive and impractical. All of the funds are tied to the address, while the public key is used to prove the ownership of the private key without revealing it.

Internally, Mesh stores the balance as integers to avoid floating point errors, where 1000 units equal 1 Mesh. As a result, the lowest denomination is 0.001. The balance is calculated directly from the full chain by scanning all historical incoming and outgoing transactions.

There are 3 types of balances which are used: verified, unverified and spendable. The verified balance is the amount of Mesh an address can spend, which is calculated by only taking into account the transactions in blocks. The unverified balance also takes into account unverified transactions which are in the mempool. On the other hand the spendable balance takes into account all unverified outgoing transactions while ignoring unverified incoming transactions, which is the amount the address can actually spend. For example, suppose these are all of the transactions tied to an address:

Verified transaction: +10

Verified transaction: -3

Unverified transaction: +10

Unverified transaction: -5

Then the verified balance would be $10 - 3 = 7$ (The verified balance is used to calculate unverified and spendable), unverified would be $7 + 10 - 5 = 12$, while spendable would be $7 - 5 = 2$. As a result, the address would expect to be able to use 12 Mesh once the unverified transactions get verified, and until verification would be able to only spend 2 more.

3. Transactions

Each transaction allows the transfer of Mesh from address A to address B. Each transaction follows the format:

`sender|receiver|amount|nonce|extraFee||publicKey||signature`

Amount, nonce and fee use denary, while the remaining fields use hexadecimal. The sender and receiver fields are both addresses. Here is an example of a transaction where address dae54932... sends 3.141 Mesh to 3006db7d...:

`dae54932...|3006db7d...|3141|75|7||0354895f...||3044022043ae...`

The nonce is a unique number assigned to each transaction, ensuring that each transaction can only be spent once, as any repeated use will contain the same nonce. The signature contains cryptographic proof of the ownership of the private key, and will change if any of the variables like sender, receiver, amount, nonce or fee change. As a result any alteration to the transaction will result in the signature being invalid.

4. Mempool and Fees

The mempool is a list of all currently unverified transactions, sorted by extra fees. The base fee for any transaction is 1 unit (0.001 Mesh), and the extra fee increases it if needed. The fee is paid by the sender, and the receiver receives the full amount. Miners select the transactions with the highest fees to maximise gains. Consequently, increasing the fee will increase the odds of the transaction getting included in a block and being verified, as well as reducing the expected time for which it stays unverified in mempool.

5. Blocks

A block consists of a header and transactions (abbreviated as txs or tx for singular). Any transactions included in a block become verified. A block is represented as:

`priorHash|merkleRoot|timestamp|nonce,[tx1, tx2, tx3, tx4...]`

The prior [hash](#) is derived from the prior block header by hashing it twice using [SHA-256](#). This process connects all blocks in a chain, so if any header is modified then all the later blocks become invalid as the prior hash will not match the actual hash. The merkle root is a hash of all transactions with SHA-256, which also prevents any transactions from modifications as they affect the header.

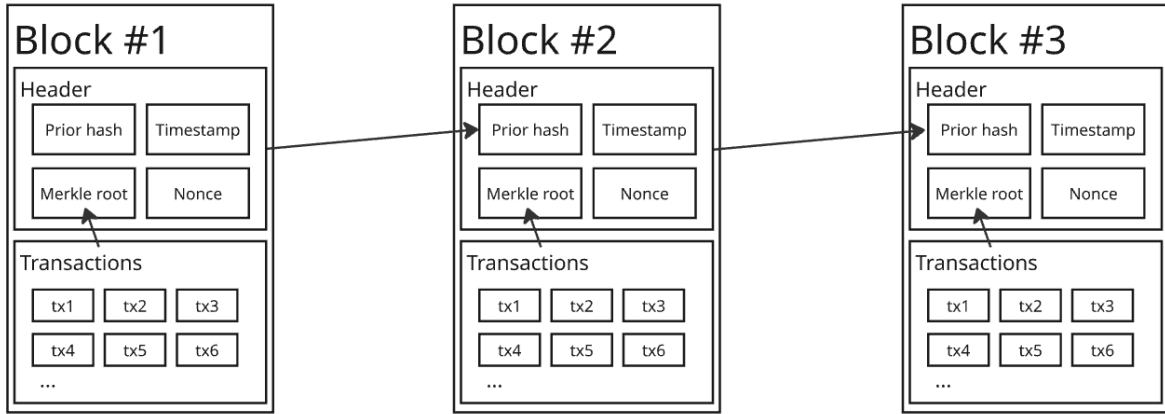


Figure 5a, a chain formed from blocks

The first transaction in a block has a special format of:

`SYSTEM|receiver|blockReward|0`

Which rewards the miner with the block reward.

6. Mining and Proof-of-Work

In order to ensure the security of transactions in blocks, creating blocks requires computational effort. Mining is the process of trying different nonces until the double hash of the header produces a small enough number. Due to the irreversible nature of SHA-256, the correct nonce cannot be calculated and can only be guessed. However verifying a nonce only requires double-hashing the header once. This process of block creation is called Proof-of-Work as the average number of computational work required to mine a block is predictable, and easily verifiable. If a miner ever finds a valid block, it gets broadcasted to the whole network.

To incentivise mining, the miner gets rewarded with the block reward, which predictably decreases over time to limit the total supply of Mesh generated. In order for the miner to add transactions to a block, fees from all transactions in the block go to the miner.

7. Block Reward

To maintain a limited supply and prevent indefinite inflation while encouraging miners to mine to secure the network, an exponential decay block reward is implemented, which is calculated by the formula:

$$\text{Block reward} = \frac{2500}{2^{\frac{b}{524288}}}$$

Figure 7a, block reward formula

Where b is the $\max(0, \text{blockHeight}-35000)$, the output is rounded up, and the output is in internal units rather than Mesh. The block reward should halve around every 5 years at the block time of 5 minutes. As a result, the resulting miner reward is calculated with:

Miner reward = Block reward + Transaction fees

8. Difficulty Adjustment

Due to the total number of miners always changing, the difficulty of mining a block has to dynamically change and adapt to ensure all miners combined can produce a block on average every 300 seconds (5 minutes). Every time the total amount of blocks (block height) is divisible by 10, difficulty has to retarget (adjust) based on the average block time over the last 10 blocks. The formula for the expected retarget can be calculated with:

$$\log_2 \left(\frac{\text{Average block time}}{300} \right)$$

Figure 8a, retarget calculation formula

Where a +1 in the output means 2x easier, and -1 in the output means 2x harder, with any adjustments over ± 1 reduced so only a quarter of the excess is applied, for example if the average time taken by each block took 9.375s compared to expected 300s, then the formula outputs -5, and since $-5 < -1$ then the remaining -4 are divided by 4 which gives -1, so difficulty will retarget by -2, which is 4x harder.

9. Block Capacity

Since all transactions have to be permanently stored by all nodes, reducing the total number of transactions reduces the total number of data each computer needs to store. By default, each block has a limit of 5 transactions after excluding the first system transaction, which increases up to 200 transactions per block depending on difficulty. For every 2.199TH (trillion hashes) added to the difficulty, 1 more transaction is allowed into the block. This naturally ensures that a small network which requires a low number of transactions limits itself to a small number of transactions which reduces computer memory required.

10. Chain Selection and Forks

In order to clearly know which chain is the true chain out of many (as well as which chain miners should extend), all nodes must follow an equivalent ruleset to all converge on the same chain out of many. The best chain is the chain with the most work, however if the node already has a chain, the competing chain must also have a greater block count to switch chains. A fork is when the same chain splits where there are two alternate versions, as shown in Figure 10a:

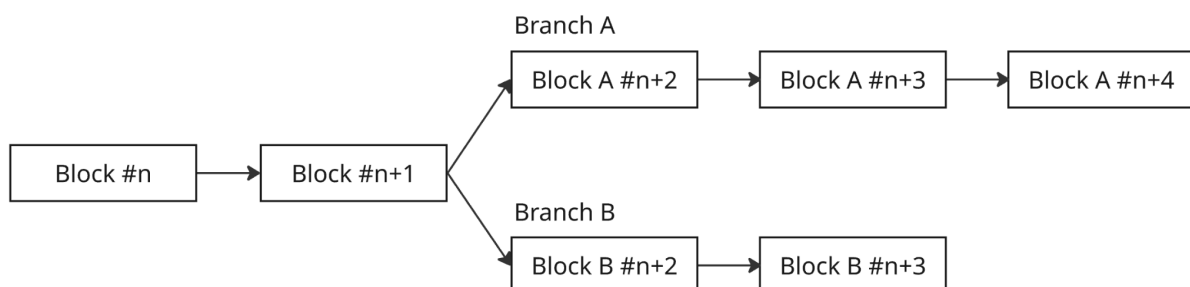


Figure 10a, competing forks

Since A is longer and had more total work, all nodes will decide to abandon branch B and will switch to A as it is a clearly better alternative, resulting in Figure 10b:

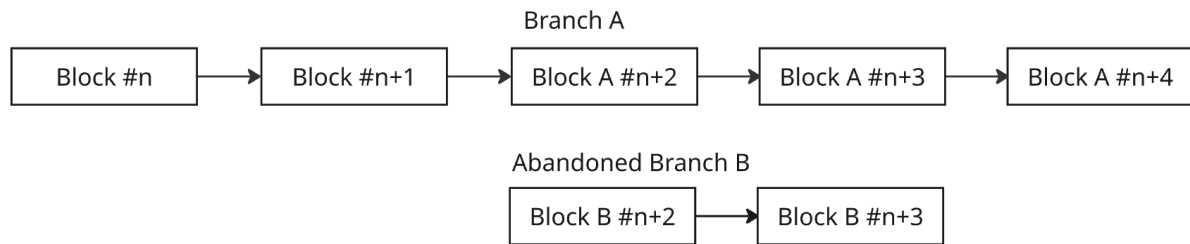


Figure 10b, branch B abandoned

As a result, all transactions and balance changes which happened over blocks in the abandoned branch are ignored, unless the same transactions got added to both blocks A and B, and most of the time the transactions will be added to both blocks.

Due to the majority of the miners being honest rather than malicious, the majority will mine on the longer chain resulting in an attacker having a very small chance of reversing history as that would require outpacing all the honest nodes, the odds of which decrease as more blocks are mined after. Hence, transaction security is in proportion with the number of valid blocks after the transaction.